FORWARD-DEPLOY GUIDELINE

Forward-Deploy Guideline — Server-Side GraphQL + Agentic Workflows + Tool Runner

For merchants, app developers, and platform teams planning the Shopify roadmap. **The 2025–2026 deprecation cliff retires the client-side / REST / Script-Editor era** — the durable forward path is server-side GraphQL orchestrated by an agentic Tool Runner. Reviewed quarterly against Shopify's published deadlines.

Audience: merchants, app developers, and platform teams planning their Shopify roadmap.

Thesis: Shopify's 2025–2026 deprecation cliff retires the *client-side / REST / Script-Editor*

---

era. The durable forward path is **server-side GraphQL workflows orchestrated by an agentic Tool Runner.** This guideline turns the deadlines into a migration plan.

As of: 2026-06-21 · **Review:** quarterly (Shopify versions sunset on a rolling schedule). All dates below are Shopify-published deadlines — plan each pillar against them.

1. THE FORCING FUNCTION (WHY NOW)

DATE	WHAT ENDS	WHAT IT FORCES
Oct 1, 2024	REST Admin API marked **legacy**	New build must be **GraphQL-first**
Apr 1, 2025	New public apps must be **GraphQL-only**	REST skills stop compounding
Jan 1, 2026	Can no longer **create** legacy custom apps	Move to managed install + token exchange
Jun 30, 2026	**Shopify Scripts removed** (payment/shipping/line-item)	Checkout logic must be **server-side Functions**, not Script Editor
Rolling (quarterly)	API versions sunset ~12 months after release	Pin a current version; automate the bump

The pattern across every row is the same: **logic that lived in the browser, in REST, or in the Script Editor is moving to the server, to GraphQL, and to declarative Functions.** Once you accept that, the question is not *whether* to go server-side — it's *what orchestrates the server-side calls*. That orchestrator is the opportunity.

2. THE SHIFT IN ONE LINE

> **Client-side scripts + REST polling → server-side GraphQL workflows invoked by an agentic Tool Runner.**

- **System of record:** Shopify, via the **GraphQL Admin API** (orders, products, customers, draft orders, returns, discounts, markets, translations).
- **Checkout/business logic: Shopify Functions** (the Scripts replacement) — declarative, server-evaluated, version-pinned.
- **Orchestration:** a **Tool Runner** — an MCP server that exposes each GraphQL/Function operation as a typed *tool*, resolves data on your own edge

(e.g. Cloudflare Workers), and holds the credentials so agents never do.

This is the same three-layer split agents already expect: **discovery → authorization → execution**, with the Tool Runner as the execution plane.

The anti-pattern this replaces

The common "before" state is a **Liquid theme stuffed with client-side JavaScript** doing the real work: cart math, eligibility rules, price/inventory display, third-party calls, and personalization all run in the browser, glued to Liquid templates. It feels fast to ship and ages into a trainwreck:

- **Logic is unauditable and duplicated** across theme files, snippets, and inline `<script>`s.
- **Secrets and API calls leak client-side**, or get proxied through fragile ad-hoc endpoints.
- **Every Shopify deprecation is a manual hunt** through templates — there is no single contract.
- **Agents can't drive it.** A browser-only flow has no server-side entry point for an AP2/UCP agent or your own chatbot to call; the business logic isn't reachable as a tool.
- **It breaks at the worst time** — checkout — because the rules live where the customer's network, device, and ad-blockers do.

Forward-deploy inverts this: the browser renders, the **server decides**. Liquid (or any front end) becomes a thin presentation layer; cart/checkout rules become **Functions**; data and side effects become **GraphQL behind the Tool Runner**. Same UX, but the logic is now typed, server-held, agent-callable, and survives the next deprecation.

---

3. MIGRATION MAP (RETIRE → FORWARD-DEPLOY)

RETIRING	FORWARD-DEPLOY TARGET
Payment / shipping / line-item **Scripts**	**Shopify Functions** (server-evaluated), invoked + monitored via the Tool Runner
REST endpoints / pagination loops	**GraphQL** queries + bulk operations; one round-trip, typed results
Client-side cart/checkout JS doing business rules	**Server-side** GraphQL (draft orders → mark-paid) + Functions
Legacy **custom app** install / pasted tokens	**Managed install + token exchange** (offline `id_token`), secrets server-side only
Per-channel bespoke integrations	**MCP tools** on a single Tool Runner, reused across chat / agents / storefront
Manual API-version chasing	Pinned current version (e.g. `2026-04`) + a CI bump + a deprecation watch

4. THE FORWARD-DEPLOY PILLARS

The migration above becomes real through nine workstreams. Each one moves logic from the client/Liquid/REST era to a **server-side GraphQL operation behind the Tool Runner**, and each is independently shippable (see **Code split**).

#	PILLAR	FORWARD-DEPLOY MOVE	RETIRES / FIXES
1	**Dawn → Horizon theme migration**	Adopt **Horizon** (sections/blocks, web-component-friendly) as a thin presentation shell over server-resolved data	Dawn's Liquid-heavy, client-JS theme doing business logic
2	**Code split**	Ship + scale each concern independently — theme, worker, extension, edge Functions; lazy-load surfaces	Monolithic theme bundles; one deploy blast radius
3	**Dynamic Catalog–PIM render**	Hydrate catalog / PDP from the **PIM** at the edge; PIM is the product source of truth, GraphQL is the read path	Product data hardcoded in Liquid; drift between feed and storefront
4	**Compliance + GID/UUID pairing in UCP**	Every entity carries the Shopify **GID** *and* a portable **UUID** so consent, audit, and the UCP funnel are traceable end-to-end	Untraceable identity; consent/omnibus/CPRA gaps; cross-system joins by guesswork
5	**Security**	Server-side secrets, managed install + token exchange, fail-closed guardrails, a rotation runbook	Pasted custom-app tokens, client-side secrets, last-writer-wins config
6	**Scaling**	Edge workers + GraphQL **bulk operations** + caching; quota-aware batching	REST polling, per-row loops, rate-limit cliffs
7	**Failover**	Pluggable sockets/rails with health checks and graceful degradation (settlement, search, translation)	Single-point integrations that take checkout down with them
8	**UI components — Wallets / omni-channel payments**	Composable wallet rails (Google Pay, Apple/Samsung, Kakao) bound to identity + an **AP2 mandate**, settling via swappable PSP **sockets** per market and channel	Bespoke per-channel checkout JS; one-rail lock-in
9	**Globalization — LLM / m2m100 / bge-m3**	Edge translation (**m2m100** + HTMLRewriter), multilingual semantic search / RAG (**bge-m3** embeddings), LLM-localized answers — one functional core serves every locale	Per-locale theme forks; English-only KB; manual translation drift

These are not sequential — they share the same spine (GraphQL record + Functions logic + Tool Runner orchestration), so a team can forward-deploy them in parallel and retire the Liquid/JS trainwreck pillar by pillar.

Date anchors (plan backward from these)

DEADLINE	DATE	PILLARS IT GATES
Legacy custom apps can't be created	**2026-01-01** *(passed)*	**5 Security** — managed install + token exchange is table stakes now
Shopify Scripts removed	**2026-06-30**	**1 Theme**, **8 Wallets/Payments**, and any line-item discount logic → must be **Functions / server-side** by this date
API version sunsets (rolling)	**quarterly** (~12 mo after release)	**5 Security**, **6 Scaling** — own the version bump + a deprecation watch
No Shopify deadline	—	**2 Code split**, **3 Catalog-PIM**, **4 GID/UUID**, **7 Failover**, **9 Globalization** — paced by *your* roadmap, but they unblock the dated ones

The only **hard external clock is 2026-06-30**. Everything checkout-touching (themes, wallets, discount rules) is forward-deployed to server-side Functions before then; the rest is sequenced to support it.

---

5. WHY SERVER-SIDE + TOOL RUNNER (NOT JUST "GRAPHQL")

1. **Agent-ready by construction.** An MCP Tool Runner is *already* the interface agents (and your own chatbot) call. Going server-side GraphQL without a tool layer just moves the spaghetti; the Tool Runner makes each operation discoverable, typed, and permission-gated.
2. **Credential containment.** Secrets, the Shopify token, payment/settlement keys live on the edge worker — never in the browser, never in the agent. (This is also the posture the deprecations push you toward: token exchange, no pasted custom-app tokens.)
3. **Durability against the next deprecation.** When an API version sunsets or a Function input changes, you update one tool implementation — not every

caller. The contract the agents see is stable; the Shopify call underneath is swappable.

4. **Cross-channel reuse.** The same `search_products` / `agentic_checkout` / `resolve_market` tool serves the chatbot, an external agent (AP2/UCP), and the storefront — one build, many surfaces.
5. **Lock-in avoidance.** A functional core of **Shopify GraphQL + Functions** plus your own orchestration displaces single-vendor middleware; the agent layer is yours, not rented.

---

6. FORWARD-DEPLOY CHECKLIST

☐ **Audit for Scripts** (Settings → Checkout → Script Editor). Any payment/shipping/line-item script must be a **Function** before **June 30, 2026**.

☐ **Inventory REST calls.** Replace with GraphQL; use **bulk operations** for large reads.

☐ **Move business logic server-side.** Cart/checkout rules → Functions; order creation → draft-order → mark-paid on the worker.

☐ **Stand up a Tool Runner** (MCP server on your edge) exposing each operation as a typed tool; keep the Shopify token + keys in worker secrets.

☐ **Adopt managed install + token exchange.** Retire pasted custom-app tokens; validate the embedded `id_token` server-side with the app's client secret.

☐ **Pin + watch the API version.** Run on a current version; add a quarterly deprecation review and a token-rotation runbook.

☐ **Expose agentic entry points.** Wrap the workflow so an agent (AP2 mandate / UCP discovery) can drive it through the Tool Runner without touching credentials.

---

7. THE ONE-SENTENCE PITCH

> Shopify is deleting the client-side era on a published timeline; **forward-deploy now to server-side GraphQL + Functions behind an agentic Tool Runner**, and every future deprecation becomes a one-file change instead of a fire drill.
